STL

> STL : Standard Template Library

- STL is basically a compilation of algorithms, containers, iterators, functions in a minimized version so you don't have to write lengthy lines of code and you can use that STL and you can get access to a container or to an algorithm.

> #include <bits/stdc++.h> → To include all the libraries in C++

> #include <bits/stdc++.h>

```
int main() {
    int a;
    std::cin>>a;
    std::cout<< a;
    return 0;
}
```

⇔ Similar

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a;
    cin>> a;
    cout << a;
    return 0;
}
```

> Functions :-

-
```
void print() {
    cout << "raj";
}
```

-
```
int sum (int a, int b) {
    return a+b;
}
int main() {
    print();
    int s = sum(1,2);   //prints
    cout << s;   // prints 6
    return 0;
}
```

> C++ STL → Divided into 4 parts :-

(i) Algorithms
(ii) Containers
(iii) Functions
(iv) Iterators

Scanned with OKEN Scanner

> pairs

```
pair <int, int> p = {1,3};
cout << p.first << " " << p.second;      // 1 3
p.first = 2;
cout << p.first;                         // 2
```

(~~Sprip~~ Mutable tuples)
(values can be changed)

```
pair<int, pair<int,int>> p = {1, {3,4}};
cout << p.first << " " << p.second.first << " " << p.second.second;
                                         // 1 3 4
```

```
pair<int, int> arr[] = {{1,2}, {3,4}, {6,5}};
cout << arr[1].second;                   // 4
```

pair <type1, type2>

Any data type → int / longlong / char / string / pair / vector

> CONTAINERS :-

> Vectors :-

```
vector<int> v;
```
→ Creates empty container {}

```
v.pushback(1);
v.emplace_back(2);
```
→ Both work in same way. (emplace_back() is faster)

```
vector< pair<int, int>> vec;
vec.pushback({1,2});
vec.emplace_back(1,2);
```
→ It automatically understands that given input is pair.

```
vector<int> v(5, 100);
```
→

100	100	100	100	100

```
vector<int> v(~~key~~ length, value);
```
→ {value, value, value, ...} (length)

```
vector<int> v(5);
```
→ {0, 0, 0, 0, 0}

Scanned with OKEN Scanner

- Using Iterator in vector :

eg- v = {20, 10, 15, ...}
it = v.begin()

```
vector<int>::iterator it = v.begin();
cout << *(it) << " ";   → 20
```

- Iterators points to the memory location where the value is stored. *(it) is used to access the value at that memory location.

```
it++;
cout << *(it) << " ";   → 10
```

```
vector<int>::iterator it = v.end();   → [Do it-- to get last element]
vector<int>::iterator it = v.rend();  → } In reverse vector
vector<int>::iterator it = v.rbegin();
```

Other Iterators → Never used (Just know it)

{20, 10, 15, 5, 7}
v.begin() ↑ it++ ; v.end() ↑ ← it--

{20, 10, 15, 5, 7}
v.rend() ↑ → it-- ; v.rbegin() ↑ ← it++

{7, 5, 15, 10, 20} → Reverse vector
v.rbegin() ↑ → it++ ; v.rend() ↑ ← it--

- cout << v[0] << " " << v.at(0); → 20 20
  cout << v.back() << " "; → 7

- ```
  for (vector<int>::iterator it = v.begin(); it != v.end(); it++){
      cout << *(it) << " ";
  }
  ```
  // 20 10 15 5 7 → printed

- You can replace vector<int>::iterator with auto at all places. → pointer / vector iterator

- ```
  for (auto it = v.begin(); it != v.end(); it++){
      cout << *(it) << " ";
  }
  ```

- ```
  for (auto x : v) {
      cout << x << " ";
  ```
  → int

auto itself understands the data type of variable.

Scanned with OKEN Scanner
  ```

eg

↳ datatype as string

// v = {10, 20, 12, 23}

- v.erase(v.begin() + 1); // {10, 12, 23}

// v = {10, 20, 12, 23, 35}

v.erase(v.begin() + 2, v.begin() + 4); // {10, 20, 35} [start, end)

↳ start ↳ end

Deletion in vector

- Insert function :-

vector <int> v(2, 100); // {100, 100}

v.insert(v.begin(), 300); // {300, 100, 100};

↳ where to add ↳ value

v.insert(v.begin() + 1, 2, 10); // {300, 10, 10, 100, 100}

↳ where to insert ↳ How many ↳ value

→ To insert a vector in vector :-

vector <int> a (2, 50); // {50, 50}

v.insert(v.begin(), a.begin(), ~~copy~~ a.end()); // {50, 50, 300, 10, 10, 100}

- // {10, 20}

cout << v.size(); // 2

- v.pop_back(); // {10}

- // v1 = {10, 20}

// v2 = {30, 40}

v1.swap(v2); // v1 = {30, 40} and v2 = {10, 20}

- v.clear(); → erases the entire vector {}

- cout << v.empty(); → true when no element is in vector otherwise false.

Scanned with OKEN Scanner

>> List :-

```
list <int> ls ;
ls.push_back(2) ;        // {2}
ls.emplace_back(4) ;     // {2,4}
ls.push_front(5) ;       // {5,2,4}
ls.emplace_front(3) ;    // {3,5,2,4}
```

→ push_front() of list is than insert() in vector/list

// rest functions are same as vector

// begin, end, rbegin, rend, clear, insert, size swap.

>> Set :-

- Set → sorted, Unique [All functions take O(log n)
- [set <int> st;]

  [st.insert(1) ; // {1}]
  [st.emplace(2); // {1,2}]
  st.insert(2); // {1,2}
  st.insert(4); // {1,2,4}
  st.insert(3) // {1,2,3,4}
- Functionality of insert in vector
- Can be used also, that only increases efficiency.
- begin(), end, rbegin(), rend(), size(), empty(), swap() are same as those of above.

// {1,2,3,4,5}

- auto it = st.find(3);
  ↳ Returns an iterator which points to 3.
- auto it = st.find(6); → If an element is not in set, then it returns st.end() always.

eg- s={1,2,4}
auto it = s.find(3); → // returns s.end();
{1,2,4}↑

- st.erase(5); // erases 5 // takes logarithmic time
- int cnt = st.count(1);
- auto it = st.find(3);
  st.erase(it); // Takes constant time
  ↳ Iterator
- // {1, 2, 3, 4, 5}
  auto it1 = st.find(2);
  auto it2 = st.find(4);
  st.erase(it1, it2) // {1, 4, 5}
  [start, end)
  Iterators
- // lower_bound() and upper_bound() function works in the same way as in vector it does.

  auto it = st.lower_bound(2);
  auto it = st.upper_bound(3);

>> Multi Set :-

- MultiSet → sorted ✓
  ~~Unique~~ ✗

// Everything is same as set.
// But multiset stores duplicate elements also.

- multiset <int> ms;
- ms.~~multiset~~.insert(1); // {1}
  ms.~~multiset~~.insert(1); // ~~{1}~~ {1,1}
  ms.~~multiset~~.insert(1); // {1,1,1}
- ms.erase(1); // all 1's erased.
- ms.erase(ms.find(1)); // erases first occurence of 1.
  ↳ Iterator (Address)
  {1,1,1} → {1,1}
- ms.erase(ms.find(1), ms.find(1)+2);
  [start, end)
  // {1,1,1} → {1}

>> Unordered_set :-

- Unordered_set → ~~sorted~~ ✗ Unique ✓

- unordered_set<int> st;

✓ lower_bound() and upper_bound() functions do not work.

✓ Rest all functions are same as before.

✓ Time complexity of all functions is O(1) (average)

→ It has better complexity than set in most cases, except in very rare cases where collision happens → O(N)

(Best) Average → O(1) mostly happens

Worst (Rare) → O(N)

>> Map :- (similar to dictionary in python)

[keys are unique]

- map< int , int > mpp;

  key data type — value data type

  map< int, pair< int, int >> mpp;

  map < pair<int,int>, int > mpp;

  map < char, int > mpp;

- Map stores unique keys in sorted order.

- map < int , int > mpp;
  mpp [1] = 2; (1 → key, 2 → value)
  mpp.emplace({3,1});
  mpp.insert({2,4});

  ⇒ // mpp = { {1,2} {2,4} {3,1} } → Sorted keys

- map<pair<int, int>, int > mpp;
  mpp[{2,3}] = 10; // mpp = { {2,3}, 10 }

Scanned with OKEN Scanner

```
cout << it.first << " " << it.second << endl;
}
// 1  2
   2  4
   3  1
```

- `cout << mpp[1];` // 2
- `cout << mpp[5];` // 0 → since No key is 5 (key 5 does not exist). So its value is null (0).
- `auto it = mpp.find(3);` → Returns the address of the pair {3,1}
  `cout << *(it).second;` // 1
  (it) → pair {3,1}
- `auto it = mpp.find(5);` → // mpp.end() is the value of it.
- `auto it = mpp.lower_bound(2);`
- `auto it = mpp.upper_bound(3);`

// erase, swap, size, empty, are same as above.

>> Multimap :-

↳ Duplicate keys
↳ sorted order

You can store {1,2} and again store {1,3}

- Everything same as map, only it can store multiple keys.
- mpp[key] can't be used here.

>> Unordered_map :-

- Same as set and unordered_set difference.
- Use as your requirement is.

Scanned with OKEN Scanner

> Sorting:- [ascending] [first, last) // {1, 3, 5, 4}

- sort(a, a+n);
  - a → first iterator
  - a+n → last iterator

  ⇒ {1, 3, 3, 4, 5}

- sort(v.begin(), v.end());
- sort(a+2, a+4); // {1, 3, 5, 4} ⇒ {1, 3, 4, 5}
  - a → a+2
  - a+4

- How to sort in descending order?

  sort(a, a+n, greater<int>); $\boxed{\text{or}}$ sort(a, v.rbegin(), v.rend())

  eg- {1, 3, 5, 4} ⇒ {5, 4, 3, 1}

- pair<int, int> arr[] = {{1,2}, {2,1}, {4,1}};

// sort it according to second element
// if second element is same, then sort it according to first element but in descending order.

⇒ {{4,1}, {2,1}, {1,2}}

sort(a, a+n, comp);
↳ Our bool function

```
bool comp(pair<int, int> p1, pair<int, int> p2) {
    if (p1.second < p2.second) return true;
    if (p1.second > p2.second) return false;

    // they are same
    if (p1.first > p2.first) return true;
    return false;
}
```

→ create your own function

- For array arr, n = arr.length() = no. of elements in arr.

  sort(arr, arr+n) or sort(arr+1, arr+3)

Scanned with OKEN Scanner

Count the number of 1's in binary form of a number :-

- int num = 7; → 32 bits: 000 ... 111

  int cnt = __builtin_popcount(num); // 3

- long long num = 16578657 8678;

  int cnt = __builtin_popcountll(num);

permutation of a string :-

- string = "312";

  sort(s.begin(), s.end()); // s = "123" → Very very important.

```
do {
    cout << s << endl;
} while (next_permutation(s.begin(), s.end()));
```

↳ goes in ascending order

s = 123
132
213
231
312
321
false (no more permutations)

if s = "231"

Only these will be printed because of

∴ Initially, s must be in sorted manner in ascending order

- Find max element :-

  int maxi = * max_element(a, a+n);

  → convert it back into element value.

  max_element → gives iterator of maximum element.

  a → start iterator

  a+n → end iterator

Scanned with OKEN Scanner